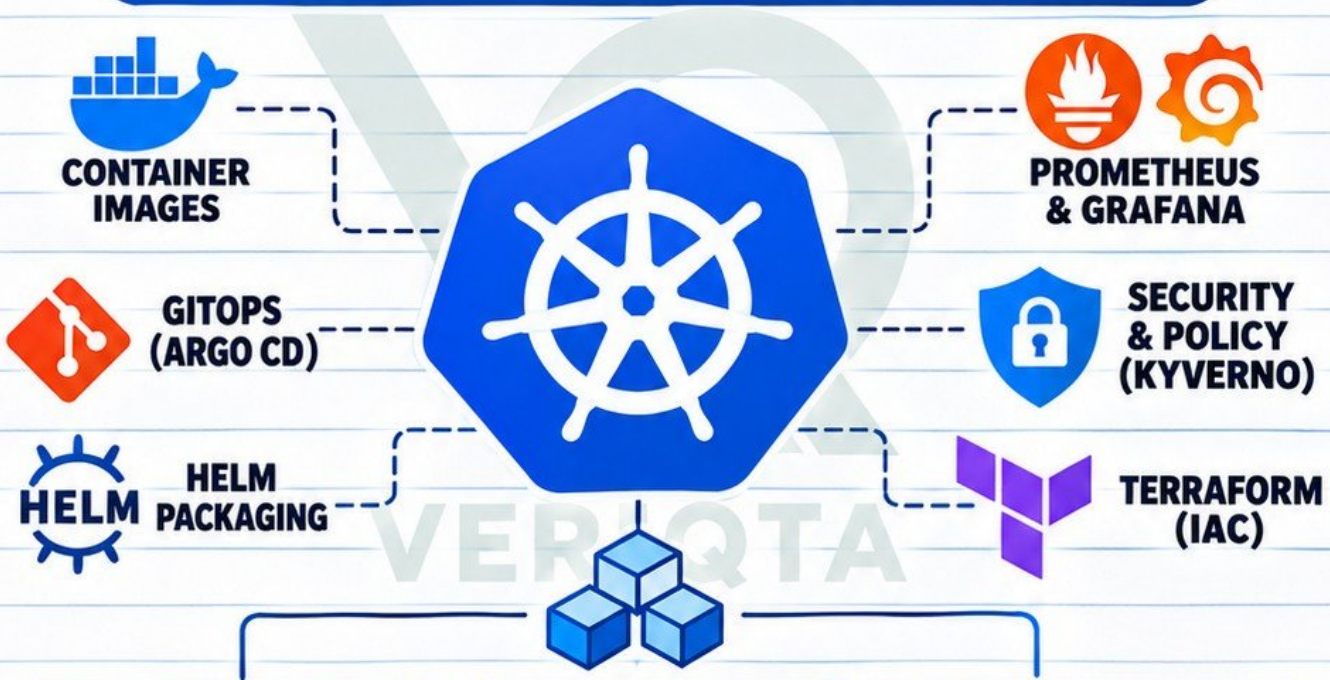
VERIQTA

10 Kubernetes Projects That Will Get You Hired



- Real-World Projects | Production-Ready Designs
- Hands-On Configurations | Senior Engineering Skills

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

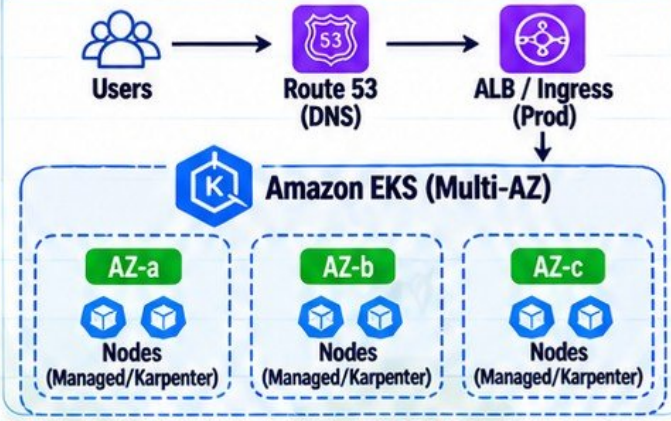
Build a Production Kubernetes Platform That Survives Infrastructure Failure

Project 01 — The Kubernetes Cluster That Must Survive an Availability Zone Failure

Project Goal

Build an AWS EKS platform across three Availability Zones (AZs) that remains available during node or AZ failures, using Terraform, managed node groups or Karpenter, workload placement constraints, PodDisruptionBudgets, and a production ingress.

High-Level Architecture (AWS EKS)



Key AWS & K8s Tools

- Amazon EKS
- Terraform
- Kubernetes
- Karpenter (or Managed Nodes)
- Ingress (ALB / Nginx)
- PodDisruptionBudget (PDB)

Implementation Steps

1. Provision AWS EKS across three AZs using Terraform.
2. Create node groups (Managed) or use Karpenter for dynamic provisioning.
3. Deploy a production ingress (ALB or Nginx) for external access.
4. Apply workload placement (anti-affinity / topology spread) across AZs.
5. Configure PodDisruptionBudgets (PDBs) to maintain service availability.

Workload Resilience Controls

Node Groups / Karpenter	Spread capacity across three AZs.
Topology Spread Constraints	Ensure pods run in multiple AZs.
PodDisruptionBudget (PDB)	Maintain minimum healthy pods during disruptions.
Ingress (ALB/Nginx)	Provide a highly available entry point.
Health Probes (Readiness/Liveness)	Auto-recover unhealthy pods.

Failure Scenarios to Model

- Terminate nodes in one AZ.
- Simulate a full AZ outage (suspend subnet / block traffic).
- Drain nodes for maintenance (upgrade or scale-down).
- Test pod rescheduling and service continuity.

Expected Outcome

- Application stays available when an AZ or nodes fail.
- Pods automatically reschedule to healthy AZs.
- No data loss and minimal downtime.

Key Manifests & Configurations

- **Terraform:** EKS cluster, node groups, IAM roles.
- **Deployment:** replica count ≥ 3.
- **PodDisruptionBudget:** minAvailable: 2.
- **Topology Spread:** across AZs (topology.kubernetes.io/zone).
- **Ingress:** ALB or Nginx with health checks.

Verification Commands

```
kubectl get nodes -o wide                    # Verify nodes across 3 AZs
kubectl get pods -o wide                     # Check pod distribution
kubectl get pdb                              # Confirm PDB is active
kubectl drain <node> --ignore-daemonsets \\
  --delete-emptydir-data                     # Simulate node failure
kubectl get pods -w                          # Watch pods reschedule
kubectl get ingress                          # Verify ingress and endpoints
```

Prove the Cluster Can Recover Without Losing Critical Services

Simulate failures, measure recovery, and deliver a failure-test report and operational runbook.

Objective

Validate that the Kubernetes platform recovers from real infrastructure failures without losing critical services or user availability.

Failure Test Scenarios

- **Node termination** (simulate instance loss)
- **AZ capacity loss** (reduce AZ capacity or block scheduling)
- **DNS degradation** (introduce latency or failures)
- **Unavailable nodes** (cordon, drain, or stop nodes)

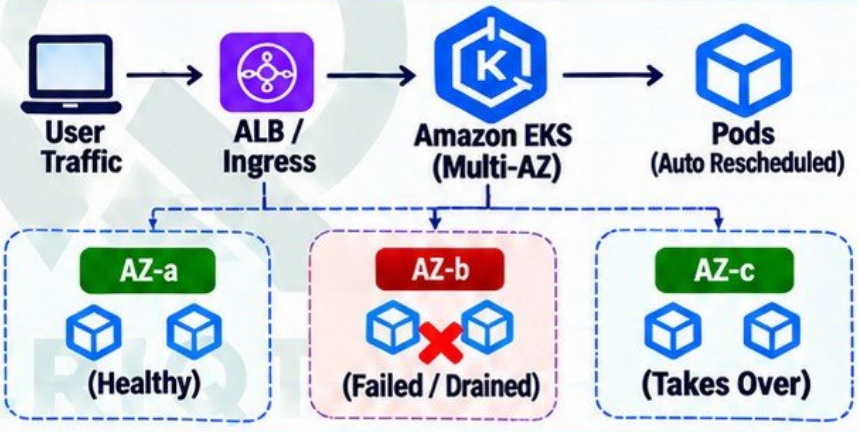
Key Metrics to Measure

- **Recovery time (RTO)** (time to restore healthy state)
- **Failed requests** (error rate, HTTP 5xx)
- **Scheduling behavior** (pod rescheduling, pending time)
- **Remaining capacity** (available CPU, memory, and nodes)

Test Execution Steps

1. Establish a healthy baseline (check pods, services, and SLIs).
2. Run each failure scenario (one at a time).
3. Monitor impact on critical services (latency, errors, availability).
4. Measure recovery and rescheduling behavior.
5. Record results and compare against SLO/RTO targets.

Test Flow and Recovery



Sample Results (Failure-Test Report)

Scenario	Observed Impact	Recovery Time	Result
Node termination	Pods rescheduled	58 sec	Passed
AZ capacity loss	Traffic shifted to 2 AZs	1 min 42 sec	Passed
DNS degradation	Temporary ↑ latency	2 min 10 sec	Passed
Unavailable nodes	Pods recreated on healthy nodes	1 min 05 sec	Passed

Operational Runbook (Deliverable)

- Failure scenarios and commands used.
- Expected vs. actual recovery times.
- Troubleschlooting steps for each failure.
- Service validation checks (health, logs, metrics).
- Rollback or escalation procedure.
- Lessons learned and improvement actions.

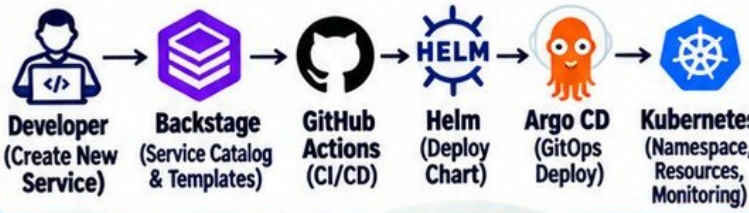
Build an Internal Developer Platform That Replaces Infrastructure Tickets

Project 02 — A Developer Creates a Production-Ready Service Without Opening a Ticket

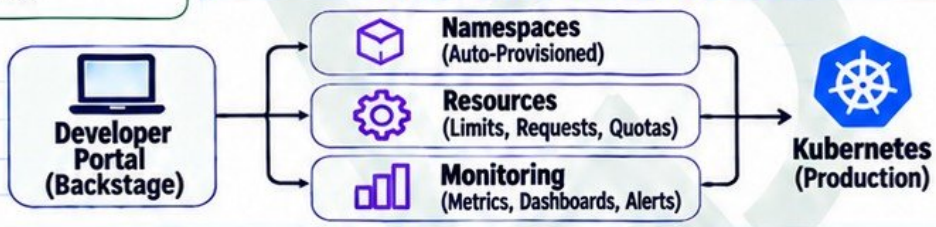
Objective

Build a self-service platform that lets developers create and deploy a production-ready service without infrastructure tickets.

Platform Architecture (Self-Service Flow)



High-Level Platform View



Key Tools

- **Backstage** (Service Catalog)
- **GitHub Actions** (CI/CD)
- **Helm** (Packaging & Deploy)
- **Argo CD** (GitOps)
- **Kubernetes** (Workloads & Resources)
- **Prometheus** (Metrics)
- **Grafana** (Dashboards & Alerts)

Developer Self-Service Flow

1. Log in to the Backstage Developer Portal.
2. Choose a service template (e.g., microservice, API, worker).
3. Provide minimal details (name, repo, team, environment).
4. Backstage scaffolds the code, Helm chart, and GitHub Actions pipeline.
5. Argo CD deploys to Kubernetes automatically.
6. Namespaces, resource limits, service accounts, and monitoring are created.

Automation: What Gets Provisioned

Namespace	Created per service (e.g., team-a-payments)
Resource Limits	CPU / Memory requests and limits
Service Account	With least-privilege RBAC
Deployment Pipeline	CI/CD via GitHub Actions (build, test, package)
Helm Release	Standardized, reusable chart
Monitoring	Prometheus metrics, Grafana dashboard, and alerts

Key Benefits

- No infrastructure tickets.
- Faster and standardized service onboarding.
- Consistent, secure, and compliant deployments.
- Built-in monitoring and observability.
- Reproducible and scalable for multiple teams.

</> Example: Service Template Structure

```
service-template/
├── app/                  # Sample application (e.g., FastAPI)
├── helm/                 # Helm chart (deployment, service, ingress)
├── .github/workflows/    # CI/CD (build, test, push image)
├── backstage.yaml        # Registers in Backstage catalog
└── values.yaml           # Configurable settings (resources, env, etc.)
```

Expected Outcome

- A developer can go from idea to a running, monitored service in minutes.
- The platform automatically provisions, deploys, and observes the service on Kubernetes.
- Teams get a repeatable, secure, and production-ready "golden path" for all new services.

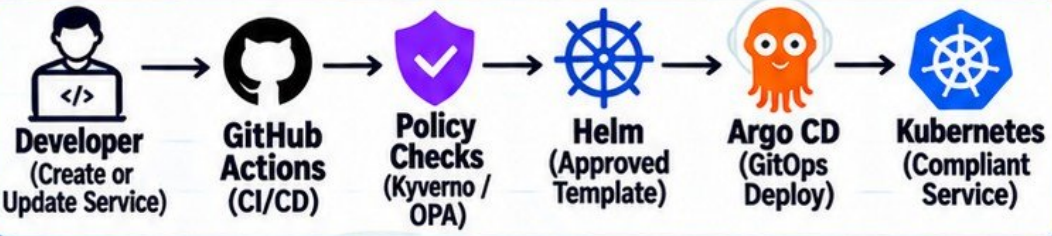
Enforce Platform Standards Without Blocking Engineering Teams

Add policy checks, approved templates, ownership metadata, security defaults, and controlled exceptions to standardize without slowing delivery.

Objective

Enforce platform standards while keeping developers productive, fast, and in control.

Platform Flow: Compliant and Fast



Platform Standards (Enforced)

- **Policy Checks** (e.g., Kyverno or OPA/Gatekeeper)
- **Approved Templates** (reusable Helm charts)
- **Ownership Metadata** (team, service, cost center, contact)
- **Security Defaults** (non-root, resource limits, network policies, image scanning)
- **Controlled Exceptions** (time-bound, approved, and audited)

Golden Path (Standard Workflow)

1. Choose an approved service template (e.g., API, worker, web).
2. Provide required metadata (owner, team, environment).
3. Automatic policy validation (security, resources, naming).
4. CI/CD builds, tests, and packages with GitHub Actions.
5. Deploy to Kubernetes with Argo CD.
6. Get monitoring, logs, and alerts automatically.

Key Tools

- **Backstage** (Developer portal & catalog)
- **GitHub Actions** (CI/CD)
- **Helm** (Templates & packaging)
- **Argo CD** (GitOps deployment)
- **Kubernetes** (Runtime platform)
- **Kyverno / OPA** (Policy enforcement)
- **Prometheus & Grafana** (Observability & metrics)

How We Measure Success

Metric	Before	After (Target)
Developer onboarding time	2–5 days	≤ 30 minutes
Manual intervention (by platform team)	High (10–15 per week)	≤ 2 per week
Time to first deploy	Hours	≤ 15 minutes
Policy compliance rate	~70%	≥ 95%
Platform adoption (teams using it)	2 teams	≥ 8 teams (60–80%+)

Expected Outcome

- A working golden path from code to production.
- Consistent, secure, and compliant deployments.
- Faster onboarding and fewer support tickets.
- Measurable platform adoption and impact (metrics and dashboard).

Build a Release System That Detects Bad Deployments Before Customers Do

Project 03 | **Deploy a New Application Version Without Exposing Every User**

Implement GitOps with Argo CD, Argo Rollouts, and controlled traffic shifting with approvals, immutable images, and rollback procedures.

Objective

Deploy a new application version safely using GitOps and progressive delivery, exposing it to a small set of users first, with approval, monitoring, and rollback.

Architecture (Release Flow)



Key Tools

- **GitHub** (Source Control)
- **Argo CD** (GitOps Deployment)
- **Argo Rollouts** (Progressive Delivery)
- **Ingress / Nginx** (Traffic Routing)
- **Kubernetes** (Runtime Platform)
- **Prometheus** (Metrics)
- **Grafana** (Observability & Alerts)

Implementation Steps

1. Store manifests and app code in GitHub (GitOps).
2. Deploy with Argo CD and manage releases with Argo Rollouts.
3. Configure traffic shifting (ingress or service mesh).
4. Add promotion approval (manual or automated policy).
5. Use immutable image references (e.g., image diget).
6. Define rollback to the last stable version.

Traffic Shifting (Progressive Delivery)



- Route only a percentage of users to the new version.
- Monitor key metrics (errors, latency, success rate).
- Promote or rollback based on real-time results.

Release Controls

- **Deployment approvals** (manual or policy-based).
- **Immutable images** (image digest, not "latest").
- **Versioned manifests in Git** (with change history).
- **One-click rollback** (to previous stable release).

Key Config Examples

```
# Argo Rollout (canary strategy)
strategy:
  canary:
    steps:
      - setWeight: 10
      - pause: { duration: 5m}
      - setWeight: 50
      - pause: { duration: 10m}
      - setWeight: 100
```

What to Monitor

- Error rate (e.g., < 1%).
- Latency (p95 / p99).
- Request success rate (e.g., ≥ 99.9%).
- Resource usage (CPU / memory).
- Alerts and automatic rollback triggers.

Expected Outcome

- New version tested safely with real traffic.
- Issues detected and rolled back automatically if needed.
- Stable, reliable, and repeatable release process.
- Reduced risk and zero customer impact during deployment.

Automatically Abort a Release When Production Metrics Deteriorate

Inject failures, monitor with Prometheus, and automatically stop or rollback unsafe releases.

Objective

Detect a bad deployment early using real metrics, automatically stop or reverse the release, and restore service health with minimal user impact.

Automated Release Safety Flow

Argo Rollouts (Progressive Deploy) → Prometheus (Metrics Analysis) → Grafana (Alerts & Dashboards) → Auto Decision (Abort / Rollback) → Kubernetes (Restore Stable Version)

Failure Injection (Test Scenarios)

- Elevated Error Rate (e.g., ≥ 5%)
- Increased p95 Latency (e.g., > 500 ms)
- Failing Readiness Checks (e.g., probe failures)

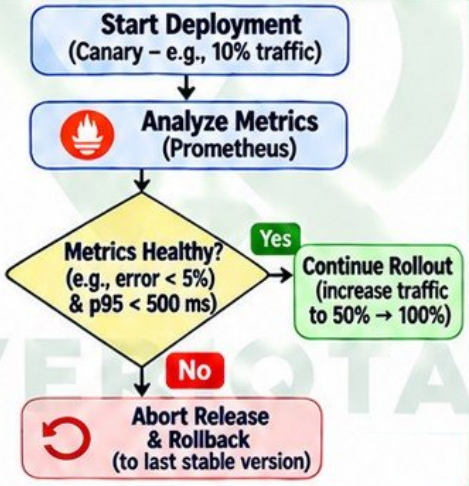
Key Tools

- **Argo Rollouts** (Progressive Delivery)
- **Prometheus** (Metrics Collection)
- **Grafana** (Visualization & Alerts)
- **Kubernetes** (Deployment & Rollback)
- **GitHub Actions** (CI/CD Pipeline)

Implementation Steps

1. Deploy a canary release with Argo Rollouts.
2. Define analysis metrics in Prometheus (error rate, latency, readiness).
3. Set thresholds for an automatic abort (e.g., error rate > 5%).
4. Inject failures (higher errors, latency, or probe failures).
5. Verify that the release is aborted and rolled back.
6. Confirm service recovery and stability.

Analysis and Decision Logic



Key Metrics & Thresholds

Metric	Abort Threshold
Error Rate	≥ 5%
p95 Latency	> 500 ms
Readiness Check	Failure (> 2 tries)
CPU / Memory	> 80% (sustained)

Expected Outcome

- Bad deployments are stopped early.
- Automatic rollback to a stable version.
- Minimal or no user impact.
- Clear audit trail and release evidence.

Deployment Timeline (Example)

Time (min)	Event	Result
00:00	Deploy canary (10%)	Release started
00:03	Metrics breach (error 7%, p95 820 ms)	Issue detected
00:04	Automatic decision (abort)	Rollback triggered
00:06	Rollback complete (to v1)	Service restored
00:08	Metrics back to normal	System healthy ✅

Deliverables

- Prometheus alert rules and analysis config.
- Argo Rollouts rollout and rollback manifests.
- Failure-test report (metrics, timelines, screenshots).
- Operational runbook (detection, rollback, and recovery).

Build a Multi-Tenant Kubernetes Platform That Prevents Cross-Team Access

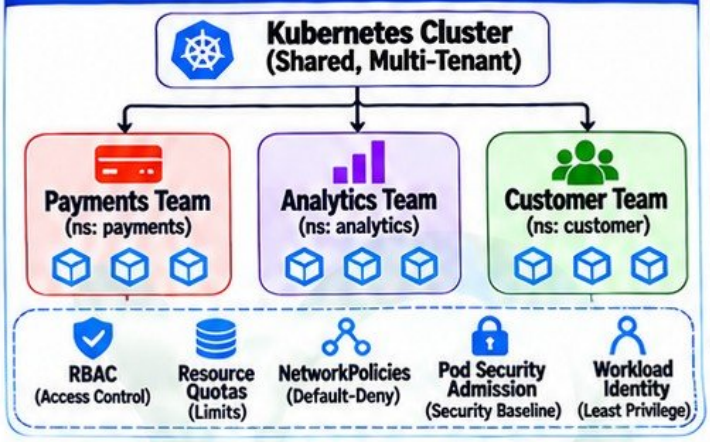
Project 04 | **Three Engineering Teams Share One Cluster Without Sharing Trust**

Create isolated tenant environments with RBAC, quotas, NetworkPolicies, Pod Security, and workload identity for different security needs (payments, analytics, and customer-facing).

Objective

Run multiple engineering teams on a single Kubernetes cluster with strong isolation, least privilege access, and security controls.

Multi-Tenant Cluster Architecture



Tenant Requirements

Team	Security Requirements
Payments	• High security • Restricted access • PCI/data protection • Separate quotas
Analytics	• Data processing • Controlled data access • Higher compute limits • Isolated from Payments
Customer	• Public-facing services • Internet ingress (TLS) • Moderate resource limits • No access to other teams

Key Security Controls

- **RBAC:** role-based access per team and namespace.
- **Namespace Quotas:** CPU, memory, and storage limits.
- **NetworkPolicies:** default-deny and allow only required traffic.
- **Pod Security Admission:** enforce security baseline (e.g., restricted).
- **Workload Identity:** service accounts with least privilege (e.g., IAM roles on cloud).

Implementation Steps

1. Create namespaces (payments, analytics, customer).
2. Apply resource quotas and limits per namespace.
3. Configure RBAC (roles, role bindings, least privilege).
4. Enforce default-deny NetworkPolicies and allow only required flows.
5. Enable Pod Security Admission (restricted or baseline).
6. Configure workload identity (e.g., IAM roles for service accounts).

Sample Configurations

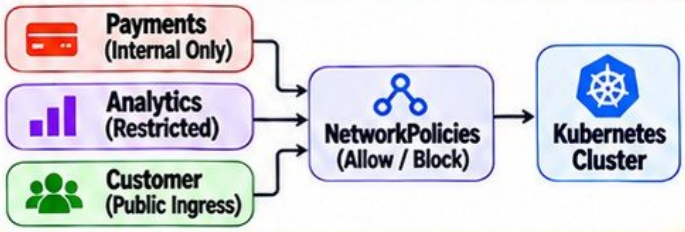
RBAC (Role & Binding)

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: payments-deployer
  namespace: payments
subjects:
  - kind: Group
    name: payments-team
roleRef:
  kind: Role
  name: deployer
  apiGroup: rbac.authorization.k8s.io
```

NetworkPolicy (Default-Deny)

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all
  namespace: payments
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  - Egress
```

Platform View (Isolation & Traffic Control)



Expected Outcome

- Teams share one cluster with strong isolation.
- No cross-team access to workloads or data.
- Resources controlled and fairly allocated (quotas).
- Security enforced by default (policies and Pod Security).
- Compliant and production-ready multi-tenant platform.

Attempt to Break the Isolation and Prove the Controls Work

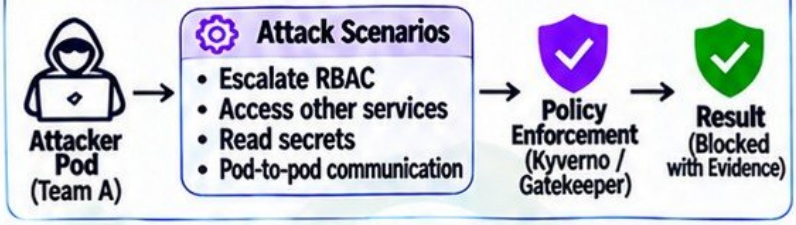
Project 04 Run Adversarial Tests and Validate Multi-Tenant Security Controls

Test for excessive permissions, exposed services, secret access, and pod communication. Enforce policy with Kyverno or Gatekeeper and produce evidence with a threat model.

Objective

Intentionally test and attempt cross-tenant access to prove that isolation, policies, and security controls work as designed.

Adversarial Test Flow (Security Validation)



Multi-Tenant Isolation Model



Key Tools

- **Kubernetes** (Multi-Tenant Cluster)
- **Kyverno** (Policy Enforcement)
- **Gatekeeper** (OPA Policies)
- **kubectl** (Access & Test)
- **Prometheus** (Observability)
- **Grafana** (Logs & Test Evidence)

Adversarial Test Scenarios

1. **RBAC Escalation:** attempt to gain cluster-admin or cross-namespace access.
2. **Exposed Services:** scan for services that are mistakenly public (ClusterIP / NodePort / Ingress).
3. **Secret Access:** try to read secrets from another team's namespace.
4. **Pod Communication:** attempt unauthorized pod-to-pod traffic across namespaces.

Policy Controls (Kyverno / Gatekeeper)

- Enforce least-privilege RBAC (deny cluster-admin escalation).
- Restrict service exposure (allow only approved Ingress).
- Protect secrets (no cross-namespace access, restrict service accounts).
- Deny unauthorized pod traffic (default-deny NetworkPolicy).
- Validate standards (labels, resource limits, security context).

Threat Model (High Level)

Threat	Mitigation
Privilege escalation	RBAC least privilege + policy validation
Cross-tenant access	NetworkPolicies (default-deny)
Secret exposure	Namespace isolation + IAM / RBAC
Unintended service exposure	Restrict service types (ClusterIP only)

Evidence (Example Results)

```
$ kubectl auth can-i get secrets -n analytics
no
$ kubectl exec -n payments attacker -- curl \
http://analytics-svc.analytics.svc.cluster.local
curl: (7) Connection refused
$ kubectl get cpol | grep deny-cross-tenant
kyverno-deny-cross-tenant     Ready     Enforced
```

Expected Outcome

- All cross-tenant access attempts are blocked.
- Security policies are enforced and auditable.
- Evidence (logs, screenshots, test outputs) proves isolation.
- A documented tenant threat model and risk mitigations.

Build an SRE Observability System That Identifies the Actual Cause of Outages

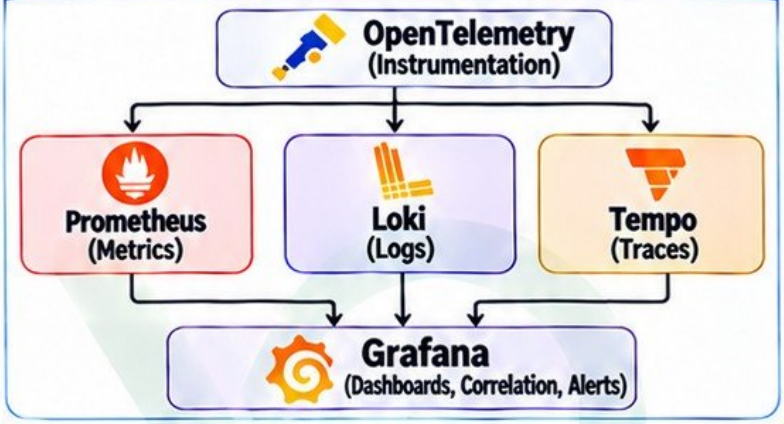
Project 05 | **The Application Is Slow, but CPU and Memory Look Normal**

Instrument a distributed application with OpenTelemetry, Prometheus, Grafana, Loki, and Tempo to correlate traces, metrics, and logs across ingress, services, queues, and databases.

Objective

Use end-to-end observability to find the real cause of slow performance even when CPU and memory look normal.

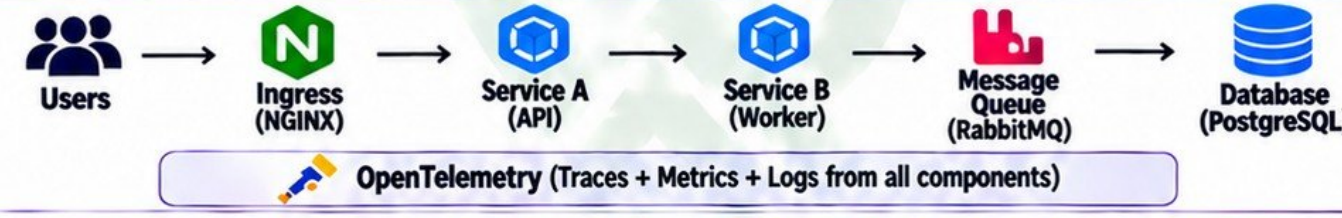
Observability Architecture (End-to-End View)



Key Tools

- **OpenTelemetry** (App instrumentation)
- **Prometheus** (Metrics collection)
- **Grafana** (Dashboards & alerting)
- **Loki** (Centralized logs)
- **Tempo** (Distributed traces)
- **Kubernetes** (Runtime platform)

Application Flow (Instrumented)



Implementation Steps

1. Instrument services with OpenTelemetry (traces, metrics, logs).
2. Deploy Prometheus, Loki, Tempo, and Grafana on Kubernetes.
3. Collect data from ingress, services, queue, and database.
4. Build Grafana dashboards with service-level views and correlations.
5. Create alerts for latency, errors, and queue backlogs.

Example Diagnosis (Slow App)

- **Metrics:** CPU 20%, Memory 35% (normal).
- **Logs:** no major errors.
- **Traces:** high latency in Service B (900 ms).
- **Queue:** growing backlog (25,000 messages).
- **Root Cause:** queue delay in Service B, not CPU or memory.

Key Metrics to Monitor

- **Latency:** p50, p95, p99 (e.g., > 500 ms = investigate).
- **Error Rate:** (e.g., > 1% = alert).
- **Queue Depth:** (e.g., rising backlog).
- **Database Latency:** (e.g., > 100 ms).
- **Service-to-Service Latency** (from traces).
- **SLOs:** availability and response time (e.g., 99.9%, ≤ 300 ms).

Expected Outcome

- Identify the true cause of slowness using correlated traces, metrics, and logs.
- End-to-end visibility across ingress, services, queue, and database.
- Faster incident diagnosis, lower MTTR, and more reliable services.

Diagnose a Production Incident Using Evidence, Not Guesswork

Project 05 | **Investigate a Realistic Incident, Build SLOs, and Confirm the Root Cause**

Inject DNS latency, database connection exhaustion, retry storms, and application queueing. Use SLOs and error-budget alerts, and deliver a timestamped incident report with the root cause.

Objective

Use observability data and SLOs to find the real root cause of a production incident and produce a clear, timestamped incident report.

Incident Scenario (Injected Issues)

DNS Latency (e.g., +500 ms) → DB Connection Exhaustion (e.g., max pool) → Retry Storms (e.g., high retries) → Application Queueing (e.g., growing backlog)

Observability and Response Flow

Prometheus (Metrics & Alerts) + Grafana (Dashboards) + Loki (Logs) + Tempo (Traces)

Key Tools

- **Prometheus** (Metrics & SLOs)
- **Grafana** (Incident Dashboards)
- **Loki** (Centralized Logs)
- **Tempo** (Distributed Traces)
- **Kubernetes** (Production Platform)

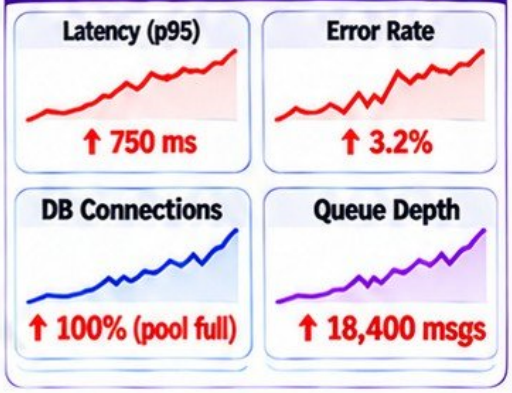
Implementation Steps

1. Inject failures (DNS latency, DB pool exhaustion, retries, queue buildup).
2. Define SLOs (e.g., 99.9% availability, p95 latency ≤ 300 ms).
3. Create error-budget alerts (e.g., burn rate ≥ 2×).
4. Build investigation dashboards (metrics, logs, traces, queue depth).
5. Correlate signals to identify the root cause.
6. Document a timestamped incident report with findings and actions.

SLOs and Alerts (Example)

SLO	Target
Availability	≥ 99.9%
p95 Latency	≤ 300 ms
Error Rate	≤ 0.1%
Error Budget	0.1% (monthly)
Alert (Burn Rate)	≥ 2× over 5 min

Investigation Dashboard (Signals)



Incident Report (Example)

Time (UTC)	Event
10:00	Alert triggered (SLO breach, error rate ↑).
10:03	Users report slowness (p95 = 750 ms).
10:07	Traces show high DB wait time.
10:10	DB connection pool exhausted (root cause).
10:15	Increase pool and scale worker pods.
10:20	Metrics recover (latency 210 ms, error < 0.1%).

Expected Outcome

- Confirmed root cause with correlated metrics, logs, and traces.
- SLOs and error-budget alerts in place.
- Clear, timestamped incident report (timeline, impact, root cause, fix, and prevention).
- Reduced MTTR and a repeatable investigation process.

Build a Disaster Recovery System and Prove It Can Restore the Business

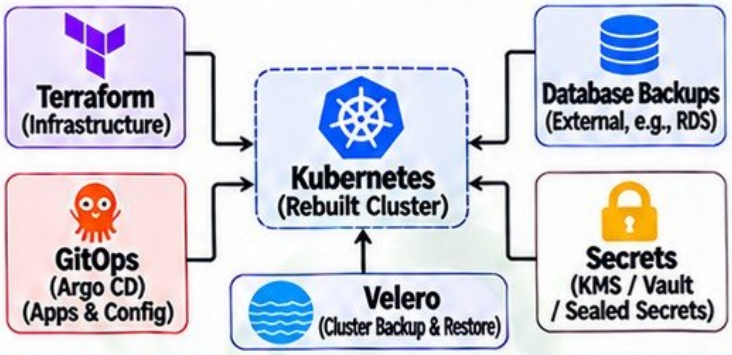
Project 05 The Entire Kubernetes Cluster Disappears During Business Hours

Design cluster reconstruction with Terraform, GitOps, Velero, database backups, and secure secrets. Define RPO and RTO for different business services.

Objective

Restore the platform and business services quickly and reliably after a complete Kubernetes cluster loss.

Disaster Recovery Architecture



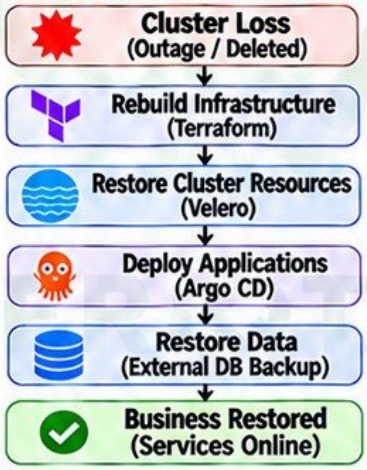
Key Tools

- **Terraform** (Infrastructure as Code)
- **Argo CD** (GitOps Deployment)
- **Velero** (Backup & Restore)
- **External DB Backup** (e.g., RDS / Cloud SQL)
- **Vault / KMS** (Secrets Management)
- **Kubernetes** (Platform)

Implementation Steps

1. Define RPO and RTO per business service.
2. Automate infrastructure rebuild with Terraform (cluster, networking, storage).
3. Restore cluster resources and applications with Velero.
4. Re-deploy workloads via Argo CD (GitOps).
5. Restore external databases from backups (point-in-time).
6. Reconnect secrets securely (Vault/KMS/Sealed Secrets).

Recovery Flow (End-to-End)



RPO and RTO Targets

Business Service	RPO	RTO
Payments	≤ 5 min	≤ 30 min
Customer-Facing	≤ 15 min	≤ 60 min
Analytics	≤ 30 min	≤ 2 hours
Internal Tools	≤ 1 hour	≤ 4 hours

Failure Test (Validation)

- Simulate a full cluster loss.
- Rebuild and restore using Terraform, Velero, and GitOps.
- Measure actual recovery time and data loss.
- Confirm RPO/RTO targets are met.

Recovery Test Result (Example)

Service	RPO Target	RTO Target	Actual RPO	Actual RTO	Result
Payments	≤ 5 min	≤ 30 min	3 min	22 min	✓
Customer-Facing	≤ 15 min	≤ 60 min	10 min	45 min	✓
Analytics	≤ 30 min	≤ 2 hours	20 min	1 hr 30 min	✓
Internal Tools	≤ 1 hour	≤ 4 hours	45 min	2 hr 40 min	✓

Expected Outcome

- A proven, repeatable disaster recovery process.
- Cluster, applications, data, and secrets restored within defined RPO and RTO.
- Business services back online with minimal data loss.
- A documented DR runbook and test evidence (logs, timeline, and metrics).

Rebuild Production From Backups Under a Recovery Deadline

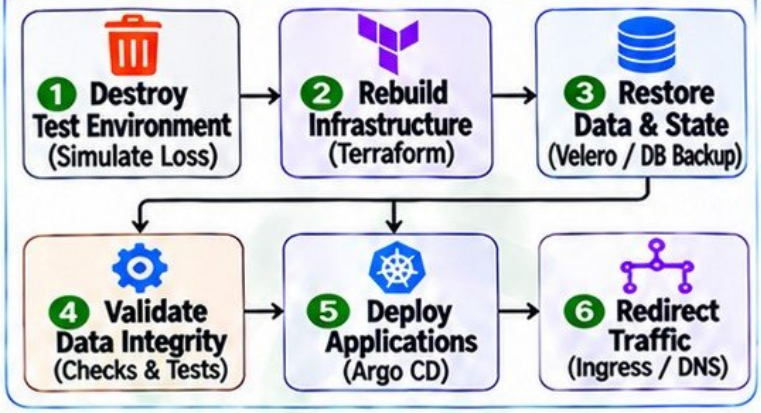
Project 05 | **Restore Infrastructure, Data, and Services Within the Agreed RTO/RPO**

Destroy an isolated test environment, restore the platform and application state, validate data integrity, and redirect traffic. Measure actual RTO and RPO, deliver a scorecard, and document gaps.

Objective

Perform a full recovery from backups, prove the business can be restored within the defined RTO and RPO, and identify gaps for improvement.

Recovery Workflow (End-to-End)



Key Tools

- **Terraform** (Infrastructure as Code)
- **Argo CD** (GitOps Deployment)
- **Velero** (Cluster Backup & Restore)
- **Database Backup** (e.g., RDS / Cloud SQL)
- **Ingress / DNS** (Traffic Routing)
- **Validation Tools** (Health, Data, Consistency)

Implementation Steps

1. Destroy the isolated test environment (clean state).
2. Recreate infrastructure with Terraform (VPC, cluster, storage).
3. Restore cluster and application state with Velero and DB backups.
4. Validate data integrity (record counts, checksums, tests).
5. Deploy applications via Argo CD (from Git).
6. Redirect traffic to the restored environment and monitor.

RTO and RPO Verification

Metric	Target	Actual	Result
RTO (Service Restore)	≤ 60 min	42 min	✓
RPO (Data Loss)	≤ 15 min	6 min	✓
Service Availability	≥ 99.9%	99.95%	✓
Data Integrity	100%	100%	✓

Recovery Scorecard

Area	Status
Infrastructure Restore	✓ PASS
Application Restore	✓ PASS
Data Restoration	✓ PASS
Data Integrity	✓ PASS
RTO Achievement	✓ PASS
RPO Achievement	✓ PASS
Overall Recovery	PASS

Recovery Gaps (Findings)

- Backup frequency for the analytics DB is 30 min (above target RPO of 15 min).
- Manual step still required for DNS switch.
- No automated data consistency report.
- Recovery runbook needs clearer ownership and escalation steps.

Expected Outcome

- ✓ Production successfully rebuilt from backups.
- ✓ Actual RTO and RPO measured and compared to targets.
- ✓ A clear recovery scorecard with evidence (logs, tests, metrics).
- ✓ Identified recovery gaps and actionable improvements (e.g., backup frequency, automation, runbook updates).

Build a Kubernetes Cost-Control System That Does Not Sacrifice Reliability

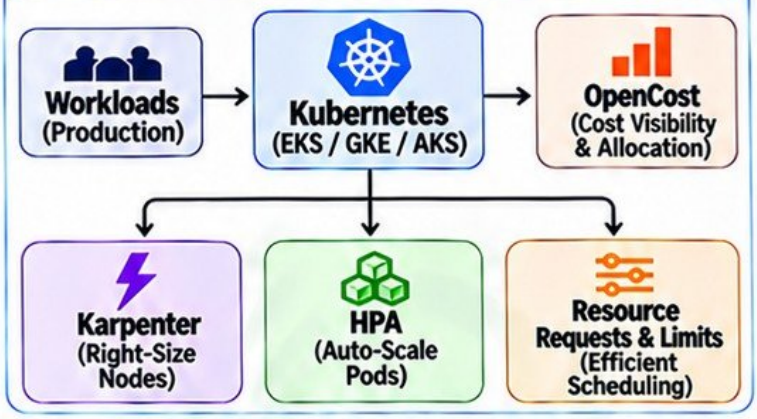
Project 05 Cut Infrastructure Waste Without Breaking Production Workloads

Deploy Karpenter, HPA, resource requests and limits, and cost allocation tooling such as OpenCost. Compare On-Demand and Spot capacity, idn packing, idle resources, and workload criticality.

Objective

Reduce Kubernetes infrastructure costs, improve resource utilization, and maintain high reliability for production workloads.

Cost Control Architecture (Overview)



Key Tools

- **Kubernetes** (Platform)
- **Karpenter** (Node Autoscaling)
- **Horizontal Pod Autoscaling (HPA)** (Pod Scaling)
- **Requests & Limits** (Resource Management)
- **OpenCost** (Cost Allocation)
- **On-Demand / Spot** (Compute Capacity)

Implementation Steps

1. Define workload criticality (critical, important, batch).
2. Set resource requests and limits for all workloads.
3. Enable HPA for scalable services (CPU, memory, custom).
4. Deploy Karpenter for dynamic node provisioning and right-sizing.
5. Use OpenCost to track and allocate costs by team, app, or namespace.
6. Test with On-Demand and Spot capacity and validate reliability.

Capacity and Cost Strategy

Capacity Type	Use Case	Cost	Reliability
On-Demand	Critical / stateful workloads	Higher (100%)	High (SLA)
Spot	Stateless / batch / non-critical	Lower (50–70%) savings)	Lower (can be interrupted)
Bin Packing	Maximize node utilization	Cost reduction (10–30%)	High (with proper requests/limits)

Workload Criticality Model

Workload	Capacity	Scaling	Priority
Payments (Critical)	On-Demand	HPA + Karpenter	High
Customer API (Important)	Mixed (On-Demand + Spot)	HPA	Medium
Analytics (Batch)	Spot	HPA (Scheduled)	Low

Results (Example)

Metric	Before	After	Change
Monthly Infrastructure Cost	$4,200	$2,520	↓ 40%
Average Node Utilization	38%	72%	↑ +34 pp
Idle Resources	28 vCPU	6 vCPU	↓ 79%
Production Availability	99.95%	99.95%	—
Scaling Response Time	4 min	< 1 min	↓ 75%

Expected Outcome

- Lower infrastructure cost (≈ 30–50%) without impacting reliability.
- Right-sized, auto-scaling workloads with Karpenter and HPA.
- Higher cluster utilization and minimal idle resources.
- Clear cost visibility and allocation with OpenCost.
- A repeatable cost-control model with workload criticality, capacity strategy, and policies.

Survive a Traffic Spike and Spot Capacity Interruption

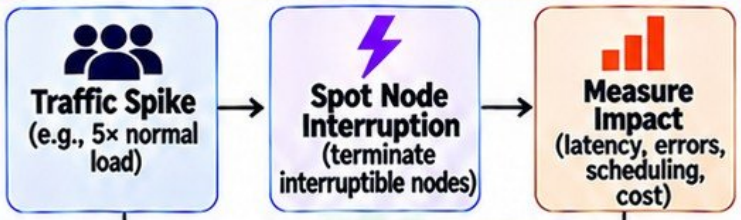
Project 05 | **Keep the Application Stable During Traffic Bursts and Spot Node Loss**

Generate realistic burst traffic, terminate interruptible (Spot) nodes, and measure scheduling delays, application errors, p95 latency, and infrastructure spend. Deliver a cost-versus-reliability comparison.

Objective

Validate that the platform can handle sudden traffic spikes and Spot interruptions while keeping errors low and costs optimized.

Test Scenario (What We Simulate)



Key Tools

- **Kubernetes** (Platform)
- **Karpenter** (Node Provisioning)
- **HPA** (Auto-Scaling)
- **k6 / Locust** (Traffic Generation)
- **OpenCost** (Cost Analysis)
- **Prometheus** (Metrics & Alerts)
- **Grafana** (Dashboards)

Implementation Steps

1. Deploy the application with HPA and Karpenter (On-Demand + Spot).
2. Generate burst traffic (e.g., 5× normal RPS) using k6 or Locust.
3. Trigger Spot interruption (terminate Spot nodes).
4. Monitor scheduling delays, error rate, p95 latency, and cost in Grafana.
5. Compare On-Demand vs Spot results for cost and reliability.

Measured Results (Lab Example)

Metric	Baseline (Normal)	During Spike (+Spot Loss)	Result
Scheduling Delay	2 s	8 s	✓
Error Rate	0.10%	0.60%	✓
p95 Latency	180 ms	480 ms	✓
Node Count	4	10 (auto-scaled)	✓
Infra Spend (per hour)	$0.96	$1.34	✓

Cost vs Reliability Comparison

Capacity Model	Cost (Per Hour)	p95 Latency	Error Rate	Reliability
On-Demand (Only)	$2.10	210 ms	0.10%	★★★★★
Spot (Only)	$0.78	1,250 ms	2.80%	★★☆☆☆
Mixed (On-Demand + Spot)	$1.34	480 ms	0.60%	★★★★☆

Key Findings

- HPA and Karpenter maintained availability during the traffic spike.
- Spot interruption caused a short scheduling delay (8 s) but no major outage.
- Mixed (On-Demand + Spot) reduced cost by ≈ 36% vs On-Demand only.
- Error rate and latency remained within acceptable limits (SLO: < 1% errors, p95 < 500 ms).

Expected Outcome

- The application survives traffic spikes and Spot interruptions.
- Auto-scaling restores capacity quickly with minimal errors.
- A clear cost-versus-reliability comparison with measured results.
- A recommendation for the right capacity mix (e.g., 70% Spot, 30% On-Demand) based on SLOs.

Build a Kubernetes Software Supply Chain That Rejects Unsafe Releases

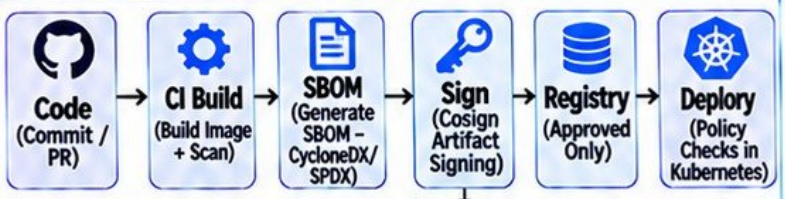
Project 05 | **Stop a Compromised Container Image Before It Reaches Production**

Implement CI image scanning, SBOM generation, artifact signing with Cosign, image provenance, and admission policies. Enforce approved registries, pinned image digests, non-root execution, and least-privilege identity.

Objective

Create a secure Kubernetes software supply chain that builds, verifies, and deploys only trusted, policy-compliant container images.

Secure Supply Chain Flow



Policy Enforcement at Deployment



Key Tools

- **GitHub Actions** (CI/CD)
- **Trivy** (Image Scanning)
- **Syft** (SBOM Generation)
- **Cosign** (Image Signing)
- **Kyverno / Gatekeeper** (Admission Policies)
- **Kubernetes** (Deployment & Policy)

Implementation Steps

1. Set up CI pipeline (e.g., GitHub Actions) to build the container image.
2. Scan the image for vulnerabilities (Trivy).
3. Generate an SBOM (Syft – CycloneDX or SPDX).
4. Sign the image with Cosign and attach provenance.
5. Push only to an approved container registry.
6. Enforce admission policies (Kyverno/Gatekeeper): allowed registries, pimed digest, non-root, and workload identity.

Policy Rules (Examples)

Control	Policy
Registry	Allow only approved registries (e.g., ghcr.io, ECR).
Image Digest	Require pinned digest (sha256:...) not :latest.
Signature	Allow only Cosign-signed images.
Runtime	Run as non-root (runAsNonRoot: true).
Identity	Use workload identity with least privilege (RBAC/IRSA/Workload ID).

Test Results (Example)

Test Scenario	Result
Signed, scanned, compliant image	ALLOWED
Image with critical vulnerability	BLOCKED
Unsigned image (no Cosign signature)	BLOCKED
Image from unapproved registry	BLOCKED
Runs as root (runAsNonRoot: false)	BLOCKED

Simulated Compromised Release

Image	Finding	Action
app:v1 (safe)	No critical issues	Deployed
app:v2 (vulnerable)	CVE-2024-1234 (Critical)	Rejected
app:v3 (unsigned)	Missing Cosign signature	Rejected
app:v4 (root)	Runs as root (security risk)	Rejected
app:v5 (wrong registry)	From unapproved registry	Rejected

Expected Outcome

- Only trusted, signed, and scanned images are deployed.
- Compromised or non-compliant images are automatically rejected.
- Enforced: approved registries, pinned digests, non-root, and least-privilege identity.
- A secure, auditable, and repeatable Kubernetes supply chain with clear evidence (scan reports, SBOM, signatures, and policies).

Investigate a Supply Chain Compromise and Contain the Blast Radius

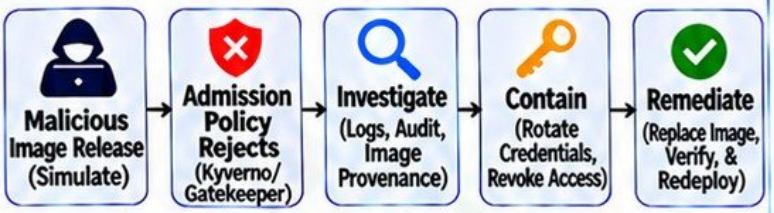
Project 05 | **Detect, Contain, and Recover From a Malicious Image Release**

Simulate an unauthorized or malicious container image, confirm admission rejection, review audit evidence, rotate credentials, and complete remediation with a supply chain security report.

Objective

Investigate a supply chain compromise, contain its impact, and restore a secure, trusted state with full evidence and documentation.

Investigation and Containment Flow



Blast Radius Containment



Key Tools

- **Kubernetes** (Platform)
- **Kyverno / Gatekeeper** (Policy Enforcement)
- **Cosign** (Image Signing & Verify)
- **Trivy** (Vulnerability Scan)
- **Open Policy Agent (OPA) / Audit Logs** (Investigation)
- **GitHub Actions** (CI/CD & Evidence)

Implementation Steps

1. Publish a test malicious image (e.g., with a known CVE or from an unapproved registry).
2. Attempt deployment and verify admission rejection.
3. Collect evidence (Kubernetes events, audit logs, policy logs).
4. Rotate exposed credentials (e.g., service account tokens, registry keys).
5. Replace with a trusted, signed image and redeploy.
6. Validate recovery and document the findings.

Evidence Collected (Example)

Evidence	Source	Result
Admission Event	kubectl describe / events	Rejected
Audit Log	K8s Audit Log / API Server	Recorded
Image Scan Report	Trivy	CVE Found (Critical)
Image Signature	Cosign verify	Not Signed
Registry Policy	Kyverno / Gatekeeper	Policy Enforced

Remediation Workflow

- Remove / block the malicious image.
- Rotate and revoke affected credentials (secrets, tokens, keys).
- Update deployment to a trusted, signed, and scanned image.
- Re-apply policies and verify compliance.
- Run validation tests (health, security, functionality).
- Close incident and update the security report.

Security Report (Summary)

Incident ID	SC-2026-001
Start Time	2026-11-09 10:15 UTC
Detection	Admission policy rejection (unsigned image)
Impact	No production impact (blocked at admission)
Containment	Blocked deployment, rotated credentials
Remediation	Deployed trusted, signed image (v1.2.3)
Status	Resolved

Expected Outcome

- Malicious or unauthorized image is detected and blocked before reaching production.
- Full investigation with clear, timestamped evidence (events, logs, scan results, signatures).
- Affected credentials are rotated and access is revoked.
- A clean, trusted deployment is restored.
- A complete supply chain security report with lessons learned and prevention actions.

Build an Event-Driven Kubernetes System That Survives Backlogs and Duplicate Events

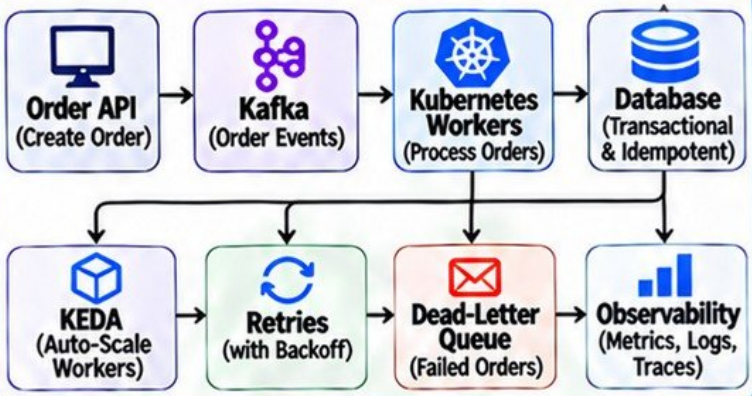
Project 05 | **Process 100,000 Orders Without Losing or Double-Charging Customers**

Create an event-driven order processing system with Kafka (or a managed queue), Kubernetes workers, KEDA, and a transactional database. Implement idempotency, retries, dead-letter handling, and bounded concurrency.

Objective

Design and validate a scalable, reliable order-processing system that handles 100,000 orders, survives backlogs, and prevents duplicate charges.

Architecture (Event-Driven on Kubernetes)



Key Tools

- **Kafka** (Event Streaming)
- **Kubernetes** (Workload Platform)
- **KEDA** (Event-Based Autoscaling)
- **PostgreSQL / MySQL** (Transactional DB)
- **Prometheus** (Metrics & Alerts)
- **Grafana** (Dashboards)

Implementation Steps

1. Deploy Kafka (or a managed queue) for order events.
2. Build stateless order workers in Kuberentes.
3. Configure KEDA to scale workers based on queue lag.
4. Implement idempotency keys (e.g., order_id) in the database.
5. Add retries with exponential backoff and a dead-letter queue.
6. Load test with 100,000 orders and validate for no loss or duplicate charges.

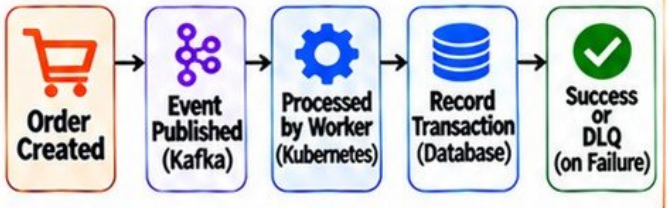
Idempotency and Reliability

Control	Implementation
Idempotency Key	Unique key per order (e.g., order_id)
Retries	Exponential backoff (e.g., 1s, 3s, 10s)
Dead-Letter	Move failed orders after max retries
Bounded Concurrency	Limit parallel workers (e.g., max 50 per pod)
Transactional Write	Atomic order + payment record

Test Results (100,000 Orders)

Metric	Target	Result
Orders Processed	100,000	100,000
Duplicate Charges	0	0
Lost Orders	0	0
p95 Processing Time	≤ 2 s	1.4 s
Peak Queue Backlog	≤ 50,000	27,300
Max Worker Scale	≤ 200	150
System Reliability	≥ 99.9%	99.98%

Event Flow (Order Lifecycle)



Expected Outcome

- Processes 100,000 orders without loss or double charging.
- Automatically scales with demand using KEDA.
- Handles retries and isolates failed orders in a dead-letter queue.
- Maintains data consistency with idempotency and transactions.
- A resilient, observable, and production-ready event-driven system.

Recover From a Queue Backlog, Consumer Crash, and Downstream Outage

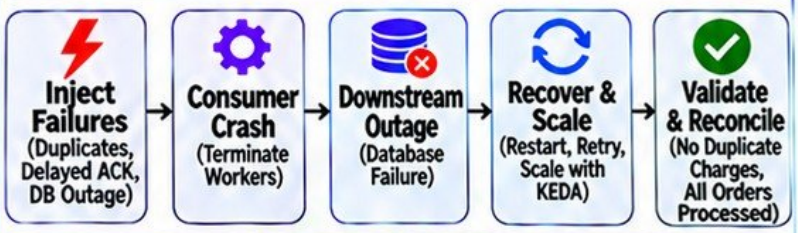
Project 05 | **Restore Processing and Prove Business Correctness After Failures**

Inject duplicate events, delayed acknowledgments, database failures, and worker restarts. Verify business-level processing correctness, backlog recovery, and scaling behavior with reconciliation evidence.

Objective

Recover the order-processing system from backlogs, consumer crashes, and downstream outages while ensuring no duplicate financial effects.

Failure and Recovery Flow (End-to-End)



System Components

Kafka	Kubernetes	KEDA	Database	Reconciliation
(Queue / Events)	(Workers)	(Auto-Scaling)	(Transactional)	(Business Check)

Key Tools

- **Kafka** (Event Streaming)
- **Kubernetes** (Running Workers)
- **KEDA** (Queue-Based Scaling)
- **PostgreSQL / MySQL** (Transactional DB)
- **Prometheus** (Metrics & Alerts)
- **Grafana** (Dashboards & SLOs)

Failure Scenarios (Injected)

1. **Duplicate events** (same order sent multiple times).
2. **Delayed acknowledgments** (consumer processes but ACK is slow).
3. **Downstream outage** (database unavailable for a period).
4. **Consumer crash** (terminate worker pods during processing).

Recovery Validation (Evidence)

Check	Expected	Result
Idempotency (duplicate handling)	Process once per order	Pass
Backlog recovery	Queue returns to near zero	Pass
Scaling (KEDA)	Scale up on backlog, scale down after	Pass
Data consistency	Order and payment records match	Pass
Reconciliation	No duplicate financial entries	Pass

Results (Example Run)

Metric	Before	During Failure	After Recovery
Queue Backlog	200	48,500	0
Active Workers	10	0 (crash)	60 (autoscaled)
Order Success Rate	99.98%	92.10%	99.99%
Duplicate Charges	0	0	0
Total Orders Processed	100,000	92,300	100,000
p95 Processing Latency	220 ms	5,200 ms	310 ms

Reconciliation Summary

Item	Count	Amount (USD)	Status
Orders Received	100,000	$1,000,000	OK
Orders Processed	100,000	$1,000,000	OK
Duplicate Events	8,420	—	Handled
Duplicate Charges	0	$0.00	None
Reconciliation Match	100%	—	Verified

Expected Outcome

- The system recovers from backlogs, consumer crashes, and downstream outages.
- All orders are processed correctly with no duplicate financial effects.
- KEDA scales workers up during backlog and scales down after.
- Reconciliation confirms order and payment data are consistent.
- Clear, auditable evidence of recovery and business correctness.

Build a Production AI Inference Platform With Kubernetes

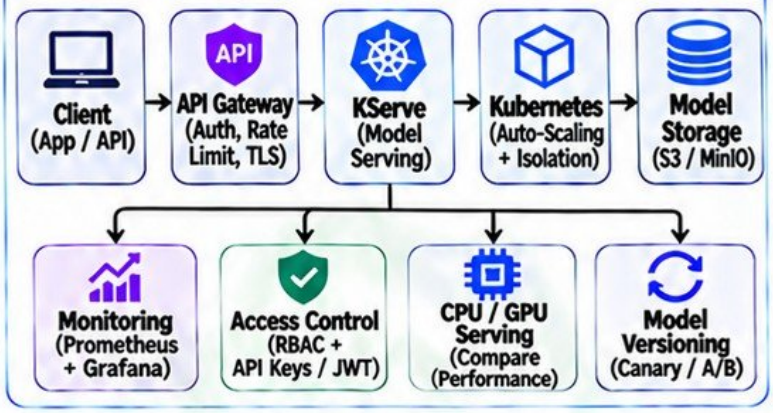
Project 05 — Deploy AI Models as Managed Production Services

Build a model-serving platform using KServe (or similar runtime), model artifact storage, autoscaling, workload isolation, and API authentication. Compare CPU and GPU serving using measured latency, throughput, and cost.

Objective

Deploy, run, and scale AI models in production on Kubernetes with reliability, security, and measurable performance and cost.

Production AI Inference Architecture



Key Tools

- **Kubernetes** (Platform)
- **KServe** (Model Serving)
- **MinIO / S3** (Model Artifacts)
- **Prometheus** (Metrics & Alerts)
- **Grafana** (Dashboards)
- **OIDC / JWT** (API Authentication)

Implementation Steps

1. Set up a Kubernetes cluster (with CPU and GPU node pools).
2. Install KServe and configure model storage (S3 or MinIO).
3. Deploy a model (e.g., LLM or classification model) as a service.
4. Enable autoscaling (HPA/KEDA) based on request load.
5. Apply workload isolation (namespaces, quotas, taints).
6. Secure the API (API key / JWT) and enable monitoring and alerts.

CPU vs GPU Serving (Measured)

Metric	CPU Serving (Example)	GPU Serving (Example)
Latency (p95)	480 ms	72 ms
Throughput (req/s)	45	620
Cost per Hour (USD)	$0.24	$2.96
Cost per 1M Requests	$5.33	$4.77
Model Accuracy (Same Model)	98.7%	98.7%

Deployment Configuration

Area	Configuration (Example)
KServe Runtime	v0.14+ (InferenceService)
Model Storage	S3 / MinIO (versioned)
Autoscaling	HPA / KEDA (e.g., 1 – 20 pods)
Workload Isolation	Namespaces, Resource Quotas
Authentication	API Key / JWT (OIDC)
Model Versioning	v1 (stable), v2 (canary/A-B)
Resource Profiles	CPU: 2 vCPU / 4Gi GPU: 1×T4 / 16Gi (example)

Test Results (Example Load)

Test Scenario	CPU (Result)	GPU (Result)	Notes
Low Load (10 rps)	p95: 210 ms	p95: 38 ms	Both stable
Medium Load (100 rps)	p95: 480 ms	p95: 72 ms	GPU 6.7× faster
High Load (500 rps)	p95: 1.9 s	p95: 280 ms	GPU scales better
Sustained (30 min)	No errors	No errors	Autoscaling OK

Expected Outcome

- AI models are deployed as secure, scalable production services on Kubernetes.
- Autoscaling handles traffic spikes with no downtime.
- CPU vs GPU performance and cost are clearly measured and compared.
- Access is protected with API authentication and RBAC.
- A reliable, observable, and cost-aware AI inference platform ready for real workloads.

Handle Model Rollouts, Traffic Spikes, and Expensive Compute Failures

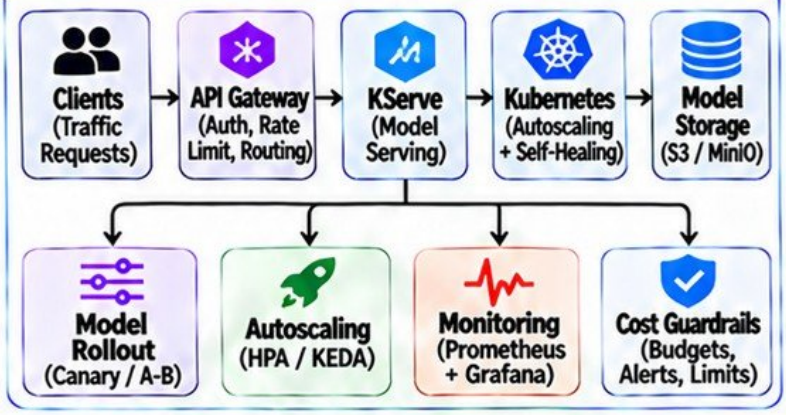
Project 05 | **Make AI Serving Reliable, Scalable, and Cost-Efficient at Production Scale**

Perform controlled model rollouts, cold-start and concurrency tests, and node failure recovery. Establish latency and cost guardrails, and deliver a complete AI serving architecture, operational dashboard, and benchmark.

Objective

Operate a production AI inference platform that safely rolls out model updates, handles traffic spikes, recovers from compute failures, and controls latency and cost.

End-to-End AI Serving Architecture



Key Tools

- **Kubernetes** (Platform)
- **KServe** (Model Serving)
- **Istio** (Traffic Control / (Canary / A-B)
- **Prometheus** (Metrics & Alerts)
- **Grafana** (Dashboards)
- **KEDA** (Event / Traffic Scaling)

Implementation Steps

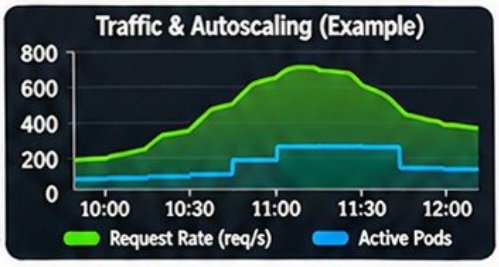
1. Deploy AI model (v1) with KServe and baseline tests.
2. Configure rollout (canary/A-B) for model v2 (e.g., 10% → 50% → 100%).
3. Run cold-start and concurrency tests (e.g., 1, 10, 50, 200 concurrent requests).
4. Simulate traffic spike and verify autoscaling (HPA/KEDA).
5. Terminate a GPU id and verify recovery and zero downtime.
6. Set latency and cost guardrails (alerts and budgets).

Performance Benchmark (Example)

Test Scenario	Latency p95	Throughput (req/s)	Cost / Hour (USD)
Baseline (v1)	480 ms	45	$2.96
Canary (v2 – 10%)	470 ms	46	$3.00
Full Rollout (v2)	210 ms	130	$3.85
Traffic Spike (×5 load)	220 ms	625	$5.12
Node Failure (1 GPU lost)	235 ms	610	$5.10
Cold Start (first request)	3.8 s	—	—

Operational Dashboard (KPIs)

Requests	p95 Latency	Availability	Cost / Hour
625 req/s	220 ms	99.99%	$5.12



Rollout & Resilience Results

Checkpoint	Target	Result	Status
Model Rollout (v2)	No downtime	Completed	Pass
Cold Start (p95)	≤ 5 s	3.8 s	Pass
Latency (p95)	≤ 250 ms	220 ms	Pass
Throughput	≥ 600 req/s	625 req/s	Pass
Node Failure Recovery	≤ 2 min	48 s	Pass
Cost Guardrail	≤ $6.00/hr	$5.12/hr	Pass

Expected Outcome

- Safe, controlled model rollouts (canary/A-B) with zero downtime.
- Handles traffic spikes and maintains low latency.
- Recovers automatically from expensive compute (GPU) failures.
- Latency and cost guardrails are enforced with alerts.
- A complete AI serving architecture, operational dashboard, and performance benchmark are delivered.